Click here to view the PDF version.

```
begin
    using Plots
    using PlutoUI
    using PlutoTeachingTools
    using LaTeXStrings
    using LinearAlgebra
    using HypertextLiteral: @htl, @htl_str
end
```

Table of Contents

Boundary value problems

Stationary heat equation in one dimension

⚠ TODO ⚠

Finite difference approximation

Error analysis

⚠ TODO ⚠

Numerical stability

Optional: Galerkin methods

Discretisation of the weak form

Example: Sine basis

Finite elements

Boundary value problems

In the previous chapter we considered initial value problems, in which one wishes to understand the behaviour of a quantity in a continious variable — usually time. Starting from the knowledge $u(a)$ at the starting point $t = a$ the methods we discussed allowed us to approximate $u(t)$ over the entire time span $t \in [a, b]$. This class of problems already covers a number of important physical phaenomena (e.g. population growth, chemical reactions, dynamics of particle systems). However, in many cases the quantity of interest not only depends on time, but also on space, such that we are effectively seeking a solution function $u(x, t)$.

Out of this class of problems we will consider a classical example, namely the heat equation, for which the unknown $u(x, t)$ is the temperature of a system at position x and time t.

Stationary heat equation in one dimension

⚠ TODO ⚠

Image

```
TODO("Image")
```

We consider a metal bar extending over the interval $[0, L]$. For a given moment in time t we denote the **temperature** at a point $x \in [0, L]$ by $u(x, t)$. The heat flux we fruther denote by $\phi(x, t)$ and allow for an external source of heat $f(x)$, e.g. by one more more heat guns pointed at the metal bar.

The rate of change of the internal heat $\frac{\partial Q}{\partial t}$ at point x of the rod is proportional to the change in temperature $\frac{\partial u}{\partial t}$, that is

$$\frac{\partial Q}{\partial t} = c\rho \frac{\partial u}{\partial t}(x, t)$$

where proportionality constants are the heat capacity c and the mass density ρ. Due to conservation of energy the change of internal heat of a slice $[x, x + \delta x]$ is equal to the incoming heat $f(x)\,\delta x + \phi(x, t)$ minus the outgoing heat $\phi(x + \delta x, t)$, such that we obtain

$$\frac{\partial Q}{\partial t}\delta x = \phi(x, t) + f(x)\,\delta x - \phi(x + \delta x, t).$$

Dividing by δx and taking the limit $\delta x \to 0$ thus yields

$$\frac{\partial Q}{\partial t} = -\frac{\partial \phi}{\partial x}(x, t) + f(x).$$

Combining with Fourier's law

$$\phi(x, t) = -k\frac{\partial u}{\partial x}(x, t)$$

where k is the thermal conductivity, we obtain the equation

$$c\rho \frac{\partial u}{\partial t}(x, t) = k\frac{\partial^2 u}{\partial x^2}(x, t) + f(x) \qquad x \in (0, L),\ t > 0$$

which is the **heat equation** in one dimension, a partial differential equation. The heat equation describes the evolution of the temperature in a system (here the metal bar) and has as its unknown the temperature distribution $u(x,t)$ at any location at any point in time.

To simplify matters in this lecture we restrict ourselves to the case of the *stationary* heat equation, i.e. the case where we assume the temperature $u(x,t)$ to be in a state where it no longer changes with time, i.e. $\frac{\partial u}{\partial t} = 0$. Dropping thus the zero terms and the dependency on t we obtain the **stationary heat equation**

$$-k\,\frac{\partial^2 u}{\partial x^2}(x) = f(x) \qquad x \in (0,L).$$

To fully specify the problem and find a unique solution we still need to indicate what happens at the extremities of the bar, i.e. at $u(0)$ and at $u(L)$. Since these two points sit at the boundary of our computational domain $[0,L]$ we usually call these conditions on u **boundary conditions**. We sketch the two most common cases:

- Consider the case where the bar is in contact with heat baths of constant temperature. A temperature α for the left-hand side bath and β for the right-hand side bath. In this case we know $u(0) = \alpha$ and $u(L) = \beta$. We obtain the **heat equation using Dirichlet boundary conditions**:

$$\begin{cases} -k\,\dfrac{\partial^2 u}{\partial x^2}(x) = f(x) \qquad x \in (0,L). \\ \qquad u(0) = \alpha \\ \qquad u(L) = \beta \end{cases} \tag{1}$$

- The bar may also be insulating at the boundary, such that no heat may be allowed to flow in or out of the extremities. In this case the boundary conditions are $0 = -k\frac{\partial u}{\partial x}(0)$ and $0 = -k\frac{\partial u}{\partial x}(L)$ (zero heat flux). We obtain a **heat equation with Neumann boundary conditions**, where the flux at the boundary is controlled:

$$\begin{cases} -k\,\dfrac{\partial^2 u}{\partial x^2}(x) = f(x) \qquad x \in (0,L). \\ \dfrac{\partial u}{\partial x}(0) = -\dfrac{\phi_L}{k} \\ \dfrac{\partial u}{\partial x}(L) = -\dfrac{\phi_R}{k} \end{cases} \tag{2}$$

  where in the insulating case $\phi_L = \phi_R = 0$. We will not consider Neumann problems any further here.

Since for such kind of problems knowing the boundary is imperative to obtain a unique solution one refers to such problems as **boundary value problems**.

Finite difference approximation

We focus on the case of a Dirichlet boundary (1) with $k = 1$. Our goal is thus to find a function $u : [0, L] \to \mathbb{R}$ with

$$\begin{cases} -\dfrac{\partial^2 u}{\partial x^2}(x) = f(x) & x \in (0, L) \\ u(0) = \alpha, \quad u(L) = \beta, \end{cases}$$

were $\alpha, \beta \in \mathbb{R}$. Similar to our approach when solving initial value problems we divide the full interval $[0, L]$ into $N + 1$ subintervals $[x_j, x_{j+1}]$ of uniform size h, i.e.

$$x_j = j\,h \quad j = 0, \ldots, N, \qquad h = \frac{L}{N+1}.$$

Our goal is thus to find approximate points u_j such that $u_j \approx u(x_j)$ at the nodes x_j.

Due to the Dirichlet boundary conditions $u(0) = \alpha$ and $u(L) = \beta$ we notice that we neccessarily need $u_0 = \alpha$ and $u_{N+1} = \beta$, such that the only unknows are the u_j with $1 \leq j \leq N$, i.e. the value at the *internal* nodes of the interval.

For these internal nodes $u(x_j)$ satisfies the equation

$$-\frac{\partial^2 u}{\partial x^2}(x_j) = f(x_j) \qquad \forall\, 1 \leq j \leq N.$$

The LHS is a second derivative, which can replace by its central finite-difference approximation (see chapter 10) to obtain

$$f(x_j) = -\frac{\partial^2 u}{\partial x^2}(x_j) \approx -\frac{u(x_{j-1}) - 2u(x_j) + u(x_{j+1})}{h^2} \qquad \forall\, 1 \leq j \leq N \tag{3}$$

or in terms of u_j:

$$\frac{-u_{j-1} + 2u_j - u_{j+1}}{h^2} = f(x_j) \qquad \forall\, 1 \leq j \leq N. \tag{4}$$

Due to our imposed boundary conditions $u_0 = \alpha$ and $u_{N+1} = \beta$, such that we can simplify the equations for $j = 1$ and $j = N$ as follows:

$$
\begin{aligned}
j = 1: \qquad & \frac{-\alpha + 2u_1 - u_2}{h^2} = f(x_1) & \Leftrightarrow \quad & \frac{2u_1 - u_2}{h^2} = f(x_1) + \frac{\alpha}{h^2} \\
j = N: \qquad & \frac{-u_{N-1} + 2u_N - \beta}{h^2} = f(x_N) & \Leftrightarrow \quad & \frac{-u_{N-1} + 2u_N}{h^2} = f(x_N) + \frac{\beta}{h^2}
\end{aligned}
$$

Putting everything together we obtain the system of equations

$$
\begin{cases}
\dfrac{2u_1 - u_2}{h^2} = f(x_1) + \dfrac{\alpha}{h^2} & j = 1 \\
\dfrac{-u_{j-1} + 2u_j - u_{j+1}}{h^2} = f(x_j) & \forall\, 2 \leq j \leq N-1 \\
\dfrac{-u_{N-1} + 2u_N}{h^2} = f(x_N) + \dfrac{\beta}{h^2} & j = N
\end{cases}
\tag{5}
$$

Introducing a vector $\mathbf{u} = (u_1, \ldots, u_N)^T \in \mathbb{R}^N$ for the unknowns and defining the *system matrix* $\mathbf{A} \in \mathbb{R}^{N \times N}$ and right-hand side $\mathbf{b} \in \mathbb{R}^N$ with

$$
\mathbf{A} = \frac{1}{h^2} \begin{pmatrix}
2 & -1 & & & \\
-1 & 2 & -1 & & \\
 & -1 & 2 & \ddots & \\
 & & \ddots & \ddots & -1 \\
 & & & -1 & 2
\end{pmatrix}
\qquad
\mathbf{b} = \begin{pmatrix}
f(x_1) + \frac{\alpha}{h^2} \\
f(x_2) \\
\vdots \\
f(x_{N-1}) \\
f(x_N) + \frac{\beta}{h^2}
\end{pmatrix}
\tag{7}
$$

we can write this as a linear system

$$
\mathbf{A}\mathbf{u} = \mathbf{b}. \tag{8}
$$

We notice that $\mathbf{A}$ is symmetric and tridiagonal. Additionally one can show $\mathbf{A}$ to be positive definite. It can therefore be efficiently solved using direct methods based on (sparse) LU factorisation or iterative approaches such as steepest-descent or the conjugate gradient method.

We summarise:

Algorithm 1: Finite differences for Dirichlet bounary value problems

Given a boundary value problem with Dirichlet boundary conditions

$$
\begin{cases}
-\dfrac{\partial^2 u}{\partial x^2}(x) = f(x) & x \in (0, L) \\
u(0) = \alpha, \quad u(L) = \beta,
\end{cases}
\tag{9}
$$

and nodes $x_j = jh$ with $j = 0, \ldots, N$ and $h = \frac{L}{N+1}$ solve the linear system $\mathbf{A}\mathbf{u} = \mathbf{b}$ with $\mathbf{A}$ and $\mathbf{b}$ given in (7) for $\mathbf{u} = (u_1, \ldots, u_N)^T$ and $u_0 = \alpha$, $u_{N+1} = \beta$.

The values $u_0, u_1, \ldots, u_{N+1}$ approximate the solution $u(x)$ at the nodal points $\{x_j\}_{j=0}^{N+1}$.

One implementation of Algorithm 1, which solves the linear system using LU factorisation is:

```
fd_dirichlet (generic function with 1 method)
```

```julia
function fd_dirichlet(f, L, α, β, N)
    # f:  Function describing the external heat source
    # L:  Length of the metal rod
    # α:  Left-hand side boundary value
    # β:  Right-hand side boundary value
    # N:  Number of nodal points for the finite-differences scheme

    h = L / (N+1)              # Step size
    x = [i * h for i in 1:N]   # Nodal points

    # Build system matrix A
    diagonal    = 2ones(N)    / h^2
    offdiagonal = -ones(N-1) / h^2
    A = SymTridiagonal(diagonal, offdiagonal)

    # Build right-hand side
    b = [f(xⱼ) for xⱼ in x]  # Evaluate function f at nodal points

    # Set terms due to boundary conditions
    b[1] = f(x[1]) + α / h^2
    b[N] = f(x[N]) + β / h^2

    # Solve problem using LU factorisation
    u = A \ b

    (; u, x, h, A, b)  # Return intermediates and results
end
```

Let us consider the example

$$\begin{cases} -\dfrac{\partial^2 u}{\partial x^2}(x) = \sin(x) & x \in (0, 2\pi) \\ u(0) = 1, \quad u(L) = 2, \end{cases}$$

i.e. the case of

```
begin
    α = 1
    β = 2
    L = 2π
    f(x) = sin(x)
end;
```

In this case the exact solution can be found analytically as

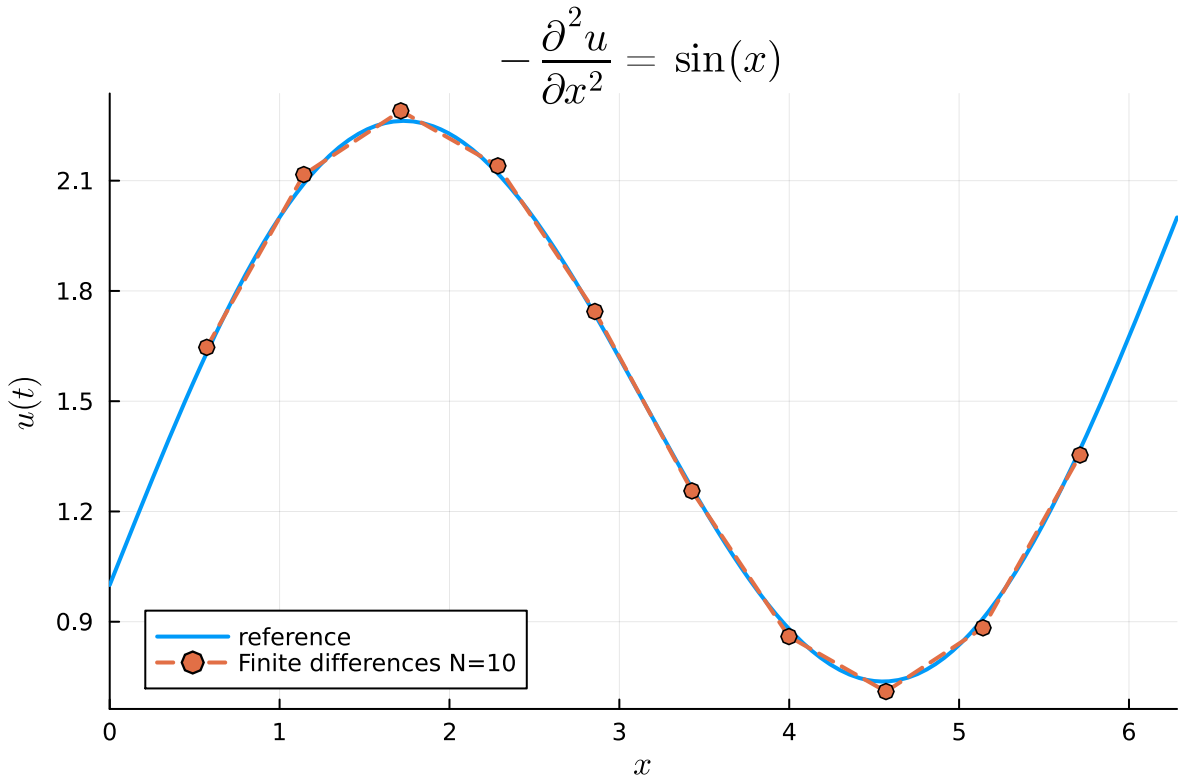
```
u_exact (generic function with 1 method)
```

```
u_exact(x) = sin(x) + x/2π + 1
```

Let's solve it with finite differences

```
res_fd = fd_dirichlet(f, L, α, β, N);
```

and plot the result:




```
let
    plot(u_exact; xlims=(0, 2π),
         label="reference", lw=2, xlabel=L"x", ylabel=L"u(t)",
         title=L"-\frac{\partial^2 u}{\partial x^2} = \sin(x)")

    plot!(res_fd.x, res_fd.u; label="Finite differences N=$N", mark=:o, lw=2,
    ls=:dash)
end
```

N = 10

Note that the plot of the finite-difference solution does not include the nodal point at the boundaries ($x = 0$ and $x = 2\pi$).

We observe that as the number of points increases, the solution becomes more and more accurate.

Note that in contrast to the initial value problems we considered in the previous chapter, such boundary value problems can always be solved, provided that the incoming heat f and the boundary values α and β are finite. More precisely we have the following result for Algorithm 1:

Theorem 1

For every $\mathbf{f} = (f(x_1), f(x_2), \dots f(x_n))^T \in \mathbb{R}^N$ and all $\alpha, \beta \in \mathbb{R}$, the system

$$\begin{cases} \dfrac{2u_1 - u_2}{h^2} = f(x_1) + \dfrac{\alpha}{h^2} & j = 1 \\ \dfrac{-u_{j-1} + 2u_j - u_{j+1}}{h^2} = f(x_j) & \forall\, 2 \le j \le N-1 \\ \dfrac{-u_{N-1} + 2u_N}{h^2} = f(x_N) + \dfrac{\beta}{h^2} & j = N \end{cases} \tag{5}$$

admits a unique solution $\mathbf{u} = (u_1, \dots, u_N)^T \in \mathbb{R}^N$ with

$$\max_{j=1,\dots,N} |u_j| \le \frac{1}{8} \max_{j=1,\dots,N} |f(x_j)| + \max(|\alpha|, |\beta|). \tag{14}$$

Colloquially speaking this Theorem ensures that the solution $\mathbf{u}$ cannot take too large values and moreover it tells us that the solution values are always controlled by the norm of the vector $\mathbf{f}$ and the boundary values α and β.

Error analysis

Employing the finite-difference scheme of Algorithm 1 to solve the Dirichlet boundary value problem (9) leads to the computed points $u_1, u_2, \dots u_N$, which approximate the function values $u(x_1), u(x_2), \dots, u(x_N)$ evaluated at the nodal points. Similar to the case of initial value problems we are interested in quantifying the error

$$|e_j| = |u(x_j) - u_j| \qquad \text{for } j = 1, \dots, N.$$

Since u_j is determined by solving (4), which thus in turn depends on the approximate values $u_{j-1} \approx u(x_{j-1})$ and $u_{j+1} \approx u(x_{j+1})$ we can identify two error contributions:

1. The local error due to employing the finite difference formula in (3) instead of the exact partial derivative.
2. The propagation of error from the neighbours u_{j-1} / u_{j+1} to u_j it self and vice versa.

We start by understanding the local error, i.e. the error due to the finite difference formula (3):

Definition: Local truncation error

Given the exact solution $u(x_j)$ of problem (9) evaluated at the nodal points $x_j = j\,h$ for $j = 0, \dots, N+1$ of the finite difference scheme of Algorithm 1, we define the **local truncation error** at the node x_j as

$$\tau_j^h = \frac{-u(x_{j-1}) + 2u(x_j) - u(x_{j+1})}{h^2} - f(x_j) \qquad j = 1, \dots, N. \tag{10}$$

That is the local truncation error is the residual of the numerical scheme (4) when we replace the approximated solution u_j by the exact solution $u(x_j)$.

⚠ TODO ⚠

Not so easy to see the point of this technical discussion.

```
TODO(md"Not so easy to see the point of this technical discussion.")
```

In our case one can show for the local truncation error:

Lemma 2: Bound on the local truncation error

If the solution u is four times differentiable the local truncation error (10) satisfies

$$\max_{j=1,\ldots,N} |\tau_j^h| \leq \frac{h^2}{12} \max_{x\in[0,L]} |u''''(x)| = \frac{h^2}{12} \|u''''\|_\infty. \tag{11}$$

▶ **Proof sketch**

Next we want to see how this resultpropagates to the global scale. We rearrange the definition of the local truncation error (10) one obtains that the exact solution satisfies the discrete problem

$$\frac{-u(x_{j-1}) + 2u(x_j) - u(x_{j+1})}{h^2} = f(x_j) + \tau_j^h \qquad j = 1, \ldots, N. \tag{12}$$

In contrast according to (4) the approximate solution satisfies

$$\frac{-u_{j-1} + 2u_j - u_{j+1}}{h^2} = f(x_j) \qquad j = 1, \ldots, N. \tag{4}$$

Subtracting (4) from (12) thus leads to

$$\frac{-e_{j-1} + 2e_j - e_{j+1}}{h^2} = \tau_j^h \qquad j = 1, \ldots, N,$$

where we used the definition of the error $e_j = u(x_j) - u_j$. At the boundary, where we set $u_0 = \alpha = u(0)$ and $u_{N+1} = \beta = u(L)$ we have $e_0 = e_{N+1} = 0$, such that the error satisfies

$$\begin{cases} \dfrac{2e_1 - e_2}{h^2} = \tau_1^h & j = 1 \\ \dfrac{-e_{j-1} + 2e_j - e_{j+1}}{h^2} = \tau_j^h & \forall\, 2 \leq j \leq N-1 \\ \dfrac{-e_{N-1} + 2e_N}{h^2} = \tau_N^h & j = N, \end{cases} \tag{13}$$

which is again a discretised Dirichlet bounary value problem (5) with the conditions that $f(x_j) = \tau_j^h$ and $\alpha = \beta = 0$.

We can therefore apply equation (14) of Theorem 1 to this problem and conclude

$$\max_{j=1,\ldots,N} |e_j| \leq \frac{1}{8} \max_{j=1,\ldots,N} |\tau_j^h|, \tag{15}$$

which relates the local truncatino error to the error at each nodal point.

Combining this result with Lemma 2 yields finally

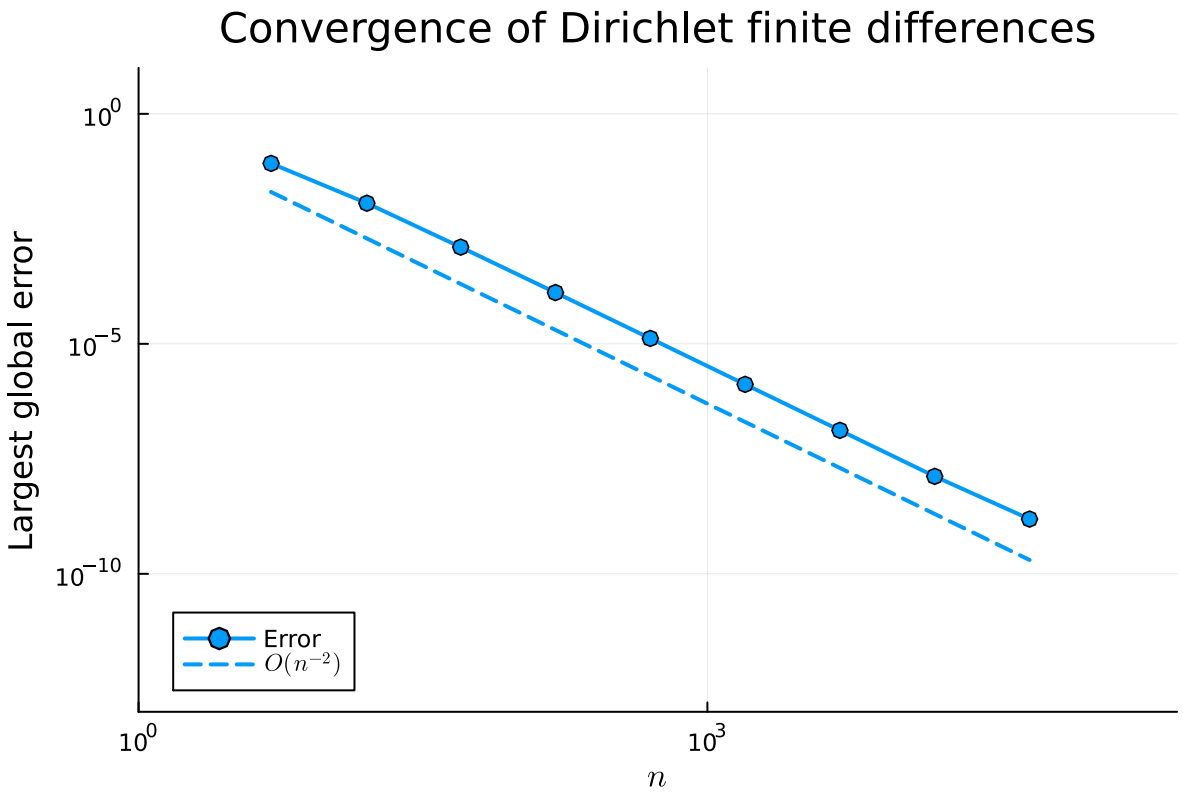
Theorem 3: Global error of finite differences

Under the assumption that the exact solution u of the Dirichlet boundary value problem (9) is four times differentiable, the finite differences scheme of Algorithm 1 converges as

$$\max_{j=1,\ldots,N} |u(x_j) - u_j| \leq Ch^2 \tag{16}$$

where $C = \frac{1}{96}\|u''''\|_\infty$. It thus achieves **quadratic convergence**.

Numerically we find indeed quadratic convergence:




```julia
let
    Ns = [ round(Int, 5 * 10^k) for k in 0:0.5:4 ]
    errors = Float64[]
    for N in Ns
        res_fd = fd_dirichlet(f, L, α, β, N)
        x = res_fd.x
        u = res_fd.u
        error = [ u_exact(x[j]) - u[j] for j in 1:length(x)]
        push!(errors, maximum(error))
    end

    p = plot(; title="Convergence of Dirichlet finite differences",
               xaxis=:log, yaxis=:log,
               xlabel=L"n", ylabel="Largest global error",
               legend=:bottomleft,
               xlims=(1, 3e5), ylims=(1e-13, 10))
    plot!(p, Ns, errors; lw=2, mark=:o, c=1, label="Error")
    plot!(Ns, 0.02(first(Ns) ./ Ns).^2, ls=:dash, lw=2, label=L"O(n^{-2})", c=1)
    p
end
```

In general we define convergence for numerical schemes for boundary value problems as follows:

Definition: Convergence order for boundary value problems

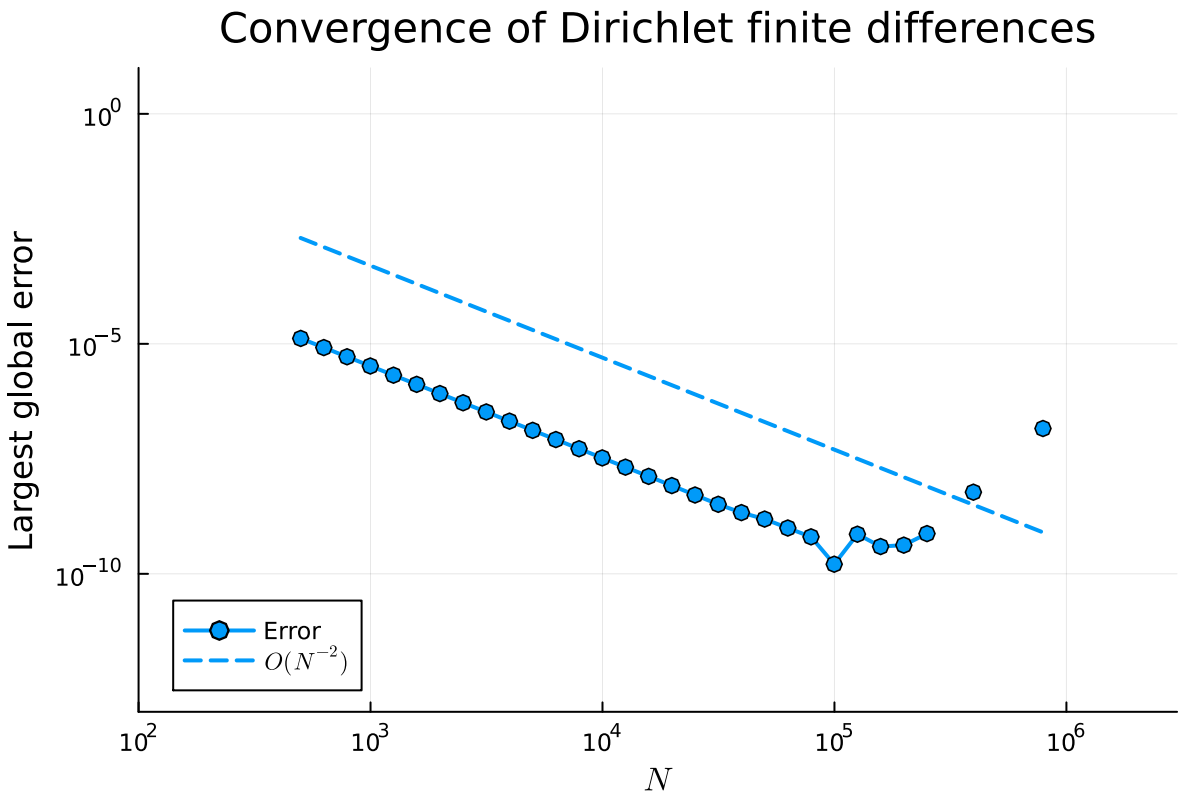
A numerical scheme approximating a boundary value problem (9) **converges with order p** if there exists a constant $C > 0$ such that

$$\max_{j=0,\ldots,N+1} |u(x_j) - u_j| \leq C\, h^p$$

provided that the solution u is sufficiently regular.

Numerical stability

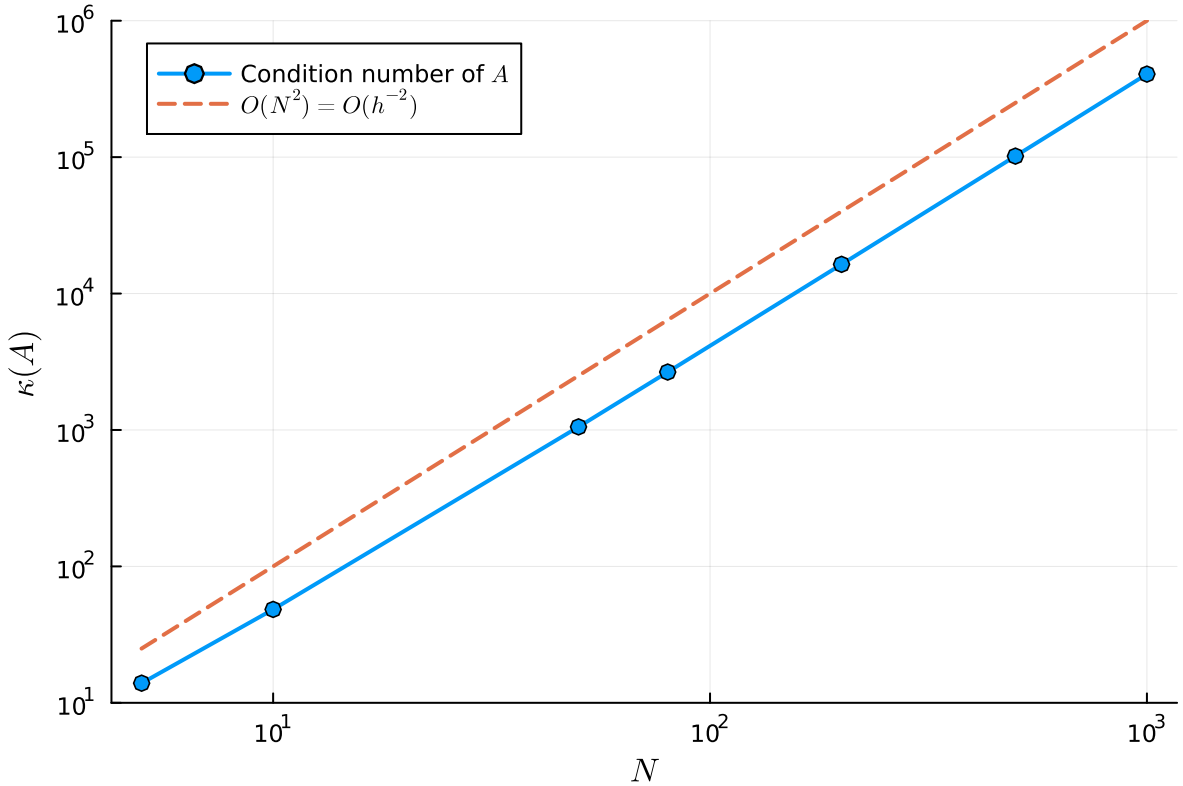
Let us push the convergence plot from above a little further:




```
let
    Ns = [ round(Int, 5 * 10^k) for k in 2:0.1:5.2 ]
    errors = Float64[]
    for N in Ns
        res_fd = fd_dirichlet(f, L, α, β, N)
        x = res_fd.x
        u = res_fd.u
        error = [ u_exact(x[j]) - u[j] for j in 1:length(x)]
        push!(errors, maximum(error))
    end

    p = plot(; title="Convergence of Dirichlet finite differences",
               xaxis=:log, yaxis=:log,
               xlabel=L"N", ylabel="Largest global error",
               legend=:bottomleft,
               xlims=(100, 3e6), ylims=(1e-13, 10))
    plot!(p, Ns, errors; lw=2, mark=:o, c=1, label="Error")
    plot!(Ns, 0.002(first(Ns) ./ Ns).^2, ls=:dash, lw=2, label=L"O(N^{-2})", c=1)
    p
end
```

While initially the convergence thus nicely follows the expected convergence curve, for larger N the convergence degrades and the error starts increasing again.

The underlying problem is related to solving the linear system $\mathbf{A}\mathbf{u} = \mathbf{b}$ in Algorithm 1. Namely, it turns out that the condition number of the system matrix $\mathbf{A}$ grows as $O(N^2) = O(h^{-2})$:




```
let
    Ns = [5, 10, 50, 80, 200, 500, 1000]
    condition_numbers = [cond(fd_dirichlet(f, L, α, β, N).A) for N in Ns]

    plot(Ns, condition_numbers; mark=:o, yaxis=:log, xaxis=:log, lw=2, xlabel=L"N",
    ylabel=L"κ(A)", legend=:topleft, label=L"Condition number of $A$", ylims=(10,
    10^6))
    plot!(Ns, Ns.^2, ls=:dash, lw=2, label=L"O(N^2) = O(h^{-2})")
end
```

As a result the finer we make the discretisation in an attempt to reduce the error to the reference, the harder it becomes to solve the system $\mathbf{A}\mathbf{u} = \mathbf{b}$.

Eventually this error will be larger than the approximation error of the numerical method, which we have quantified in Theorem 3. The result is, that the overall observed error of the numerical procedure increases again.

Optional: Galerkin methods

Let us return to the general heat equation using Dirichlet boundary conditions:

$$\begin{cases} -k\,\dfrac{\partial^2 u}{\partial x^2}(x) = f(x) \qquad x \in (0, L). \\ \qquad u(0) = \alpha, \quad u(L) = \beta \end{cases} \tag{17}$$

We saw that with finite differences the increasing condition number of the system matrix $\mathbf{A}$ with increasing number of nodal points N effectively puts a bound on the minimal error — in the above example around an error of 10^{-10}.

In this section we will develop an alternative approach to solve boundary value problems, which is more general. For this let us assume we are given a generic smooth function $\psi(x)$, the so-called **test function**, for which we will additionally assume that it vanishes at the boundary, i.e. $\psi(0) = \psi(L) = 0$. If we multiply the first line of (17) by this function and integrate over the full computational domain $[0, L]$, this yields:

$$\begin{aligned} \int_0^L f(x)\psi(x)\,dx &= \int_0^L -k\,u''(x)\psi(x)\,dx \\ &\overset{(*)}{=} [-k\,u'(x)\psi(x)]_0^L + \int_0^L ku'(x)\psi'(x)\,dx \\ &= \int_0^L ku'(x)\psi'(x)\,dx \end{aligned} \tag{18}$$

where we used partial integration in step $(*)$ and the property $\psi(0) = \psi(L) = 0$ in the final step.

Let us define the set of all test functions as

$$V_0 = \left\{ \psi : [0, L] \to \mathbb{R} \;\middle|\; \psi \text{ smooth and } \psi(0) = \psi(L) = 0 \right\}.$$

If $u : [0, L] \to \mathbb{R}$ is a solution to (17) than our discussion implies that (18) is satisfied for all functions from V_0:

$$\begin{cases} \displaystyle\int_0^L k\,u'(x)\,\psi'(x)\,dx = \int_0^L f(x)\,\psi(x)\,dx \qquad \forall\,\psi \in V_0 \\ \qquad u(0) = \alpha, \quad u(L) = \beta \end{cases} \tag{19}$$

Equation (19) is known as the **weak form** of the 1D heat equation equation (17) and solving it is interesting in its own right:

Definition: Weak form and weak solution

If a function $u : [0, L] \to \mathbb{R}$ satisfies

$$\begin{cases} \displaystyle\int_0^L k\,u'(x)\,\psi'(x)\,dx = \int_0^L f(x)\,\psi(x)\,dx \qquad \forall\,\psi \in V_0 \\ u(0) = \alpha, \quad u(L) = \beta, \end{cases}$$

for all choices of the test function ψ we call u a **weak solution** of the boundary value problem (17).

The original problem (17) is additionally called the **strong form** of the heat equation BVP and its solution u the **strong solution**.

Observation: Strong and weak solutions

If u is a solution to the strong form (17) of a BVP, than it is also a solution to the weak form (19). However, the converse is in general not true.

The weak form (19) might look unusual at first sight and one might wonder why we even consider it, especially since it seems to produce solutions, which do not satisfy (17). However, it turns out that for many contexts in physics and mathematics the weak problem can be considered to more fundamental than the strong form (17). For example in the atomistic modelling of materials (17) can lead to unphysical artifacts in models, while (19) does not.

Discretisation of the weak form

In order to solve (19) our goal is to rewrite the problem in the form of linear algebra — vectors and matrices. For simplicity we will only consider the case $\alpha = \beta = 0$ in this section, i.e. we restrict ourselves to the **heat equation with a zero Dirichlet boundary**

$$\begin{cases} -k\,\dfrac{\partial^2 u}{\partial x^2}(x) = f(x) \qquad x \in (0, L). \\ u(0) = u(L) = 0 \end{cases} \tag{20}$$

and **corresponding weak form**

$$\begin{cases} \displaystyle\int_0^L k\,u'(x)\,\psi'(x)\,dx = \int_0^L f(x)\,\psi(x)\,dx \qquad \forall\,\psi \in V_0 \\ u(0) = u(L) = 0. \end{cases} \tag{21}$$

One difficulty in this equation is that the condition $\forall \psi \in V_0$ corresponds to an infinite number of constraints to satisfy (as the set V_0 has infinitely many members). The first idea is to approximate this condition by satisfying only finitely many constraints.

To do so we assume that ψ can be written as a linear combination

$$\psi(x) = \sum_{i=1}^{m} \xi_i \, \varphi_i(x) \tag{22}$$

where $\varphi_1, \varphi_2, \ldots, \varphi_m$ is a selection of m linearly independent functions from V_0, i.e. which each satisfy $\varphi_i(0) = \varphi_i(L) = 0$. Inserting (22) into the first line of (21) leads to

$$\begin{aligned} 0 &= \int_0^L k\, u'(x)\, \psi'(x) - f(x)\psi(x)\, dx \\ &= \sum_{i=1}^{m} \xi_i \left[\int_0^L k\, u'(x)\, \varphi_i'(x) - f(x)\, \varphi_i(x)\, dx \right] \end{aligned}$$

One way to satisfy this equation is by requiring the term in the bracket to be zero for all our selected functions φ_i, that is

$$\forall i = 1, \ldots, m: \qquad \int_0^L k\, u'(x)\, \varphi_i'(x)\, dx = \int_0^L f(x)\, \varphi_i(x)\, dx \tag{23}$$

Since the φ_i are linearly independent, this can be shown to be *the only* solution. The values of the constants ξ_i are thus not relevant provided that we satisfy (23) we do not need to consider them any further.

With only a finite number of constraints to satisfy in (21) we will also approximate the solution $u(x)$ as a linear combination of finitely many functions. A natural choices is to employ the same selection of functions that we used for ψ leading to

$$u(x) = \sum_{j=1}^{m} c_j \varphi_j(x). \tag{24}$$

Inserting (24) into (23) we obtain

$$\forall i = 1, \ldots, m: \qquad \int_0^L k \sum_{j=1}^{m} c_j \varphi_j'(x)\, \varphi_i'(x)\, dx = \int_0^L f(x)\, \varphi_i(x)\, dx$$

or arranged differently

$$\forall i = 1, \ldots, m: \qquad \sum_{j=1}^{m} c_j \int_0^L k\, \varphi_j'(x)\, \varphi_i'(x)\, dx = \int_0^L f(x)\, \varphi_i(x)\, dx \tag{25}$$

Since $\varphi_j(0) = \varphi_j(L) = 0$ an immediate consequence is that such a u satisfies the boundary condition $u(0) = u(L) = 0$ independent of the choice of the coefficients c_j. Solving equation (25) therefore automatically ensures that the second line of the weak problem (21) is solved. Moreover following from our discussion it additionally represents a finite-dimensional approximation to the first line of (21). Since (25) essentially encodes the conditions on the numerical solution, it is called:

Definition: Galerkin conditions

Given a basis $\varphi_1, \varphi_2, \ldots, \varphi_m$ of m linearly independent functions from the test function set V_0, the linear combination $u = \sum_{j=1}^m c_j \varphi_j$ is an approximate solution to the weak problem (21) if the coefficients $c_1, c_2, \ldots, c_m$ satisfy the **Galerkin conditions**

$$\forall i = 1, \ldots, m: \qquad \sum_{j=1}^m c_j \int_0^L k\, \varphi_j'(x)\, \varphi_i'(x)\, dx = \int_0^L f(x)\, \varphi_i(x)\, dx$$

Collecting the unknown coefficients into a vector $\mathbf{c} = (c_1, c_2, \ldots, c_m)^T \in \mathbb{R}^m$ and introducing the matrix $\mathbf{A} \in \mathbb{R}^{m \times m}$ and the vector $\mathbf{f} \in \mathbb{R}^m$ with elements

$$A_{ij} = \int_0^L k\, \varphi_i'(x)\, \varphi_j'(x)\, dx, \qquad f_i = \int_0^L f(x)\, \varphi_i(x)\, dx$$

we can also write the Galerkin conditions more conveniently as the linear system

$$\mathbf{A}\mathbf{c} = \mathbf{f}, \tag{26}$$

which one needs to solve for $\mathbf{c}$.

Example: Sine basis

Let us consider the Dirichlet heat equation

$$\begin{cases} -\dfrac{\partial^2 u}{\partial x^2}(x) = x & x \in (0, \pi) \\ u(0) = u(L) = 0, \end{cases}$$

i.e. where $k = 1$, $L = \pi$ and $f(x) = x$.

For the basis functions φ_j we select the sine basis

$$\varphi_j(x) = \sin(j\, x) \qquad \text{for } j = 1, \ldots, m.$$

It is easy to see that all of them satisfy $\varphi_j(0) = \varphi_j(2\pi) = 0$. Using a little algebra the entries of $\mathbf{A}$ and $\mathbf{f}$ in equation (26) as

$$\begin{aligned} A_{ij} &= \int_0^\pi \frac{d}{dx}\big(\sin(i\,x)\big)\,\frac{d}{dx}\big(\sin(j\,x)\big)\,dx \\ &= i\,j\int_0^\pi \cos(i\,x)\cos(j\,x)\,dx = \frac{\pi\,i^2}{2}\,\delta_{ij} = \begin{cases} \frac{\pi\,i^2}{2} & i = j \\ 0 & i \neq j \end{cases} \end{aligned}$$

$$f_i = \int_0^\pi x\sin(i\,x)\,dx = \frac{(-1)^{i+1}\,\pi}{i}$$

For example for $m = 5$ we find the matrix and vector

$$\mathbf{A} = \frac{\pi}{2}\begin{pmatrix} 1 & & & & \\ & 4 & & & \\ & & 9 & & \\ & & & 16 & \\ & & & & 25 \end{pmatrix} \qquad \mathbf{f} = \pi\begin{pmatrix} 1 \\ -\frac{1}{2} \\ \frac{1}{3} \\ -\frac{1}{4} \\ \frac{1}{5} \end{pmatrix}.$$

Since $\mathbf{A}$ is diagonal from this we can directly compute the coefficient vector $\mathbf{c}$ as

$$\mathbf{c} = \begin{pmatrix} 2 \\ -\frac{2}{8} \\ \frac{2}{27} \\ -\frac{2}{64} \\ \frac{2}{125} \end{pmatrix}$$

and the approximate solution becomes

$$u(x) = 2\sin(x) - \frac{2}{8}\sin(2x) + \frac{2}{27}\sin(3x) - \frac{2}{64}\sin(4x) + \frac{2}{125}\sin(5x).$$

Generalising to arbitrary m we find the solution coefficients

$$c_i = \frac{f_i}{A_{ii}} = \frac{2\,(-1)^{i+1}}{i^3} \qquad \forall\, i = 1, \ldots, m$$

leading to the solution function

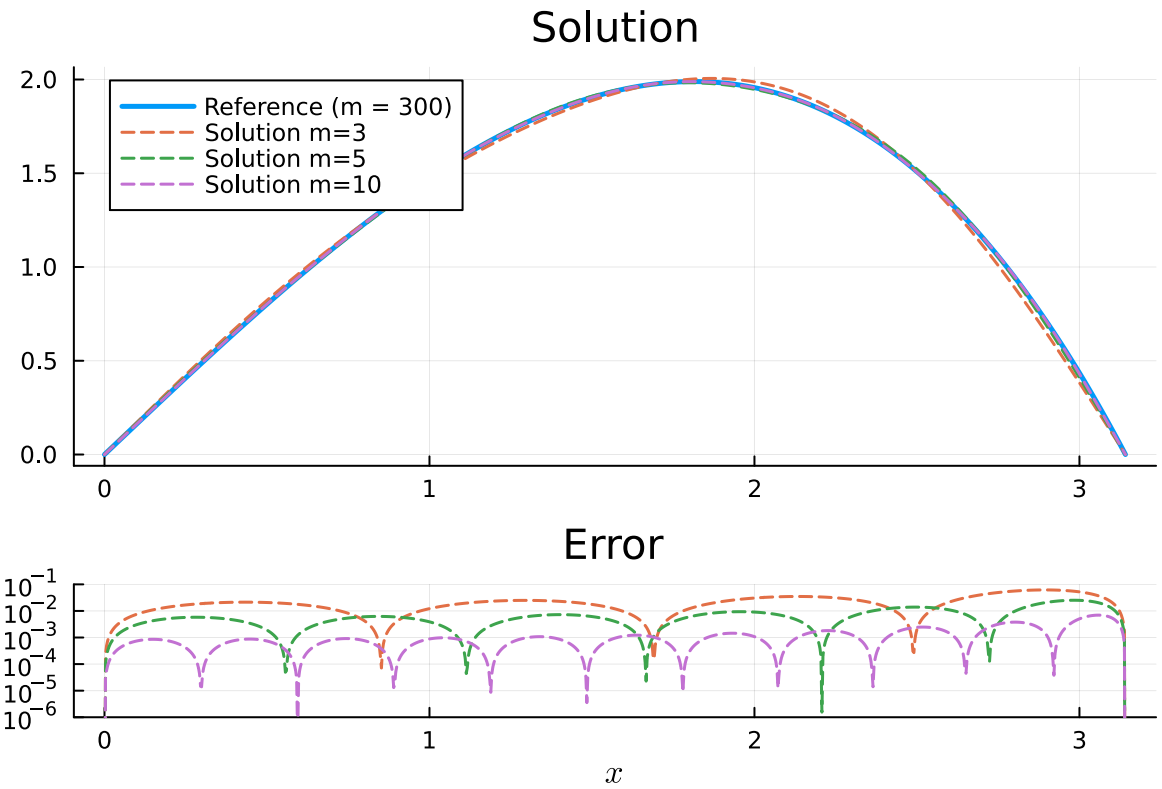
$$u(x) = \sum_{i=1}^{m} \frac{2\,(-1)^{i+1}}{i^3} \sin(i\,x), \tag{27}$$

which can be implemented as:

```
sine_solution (generic function with 1 method)
```

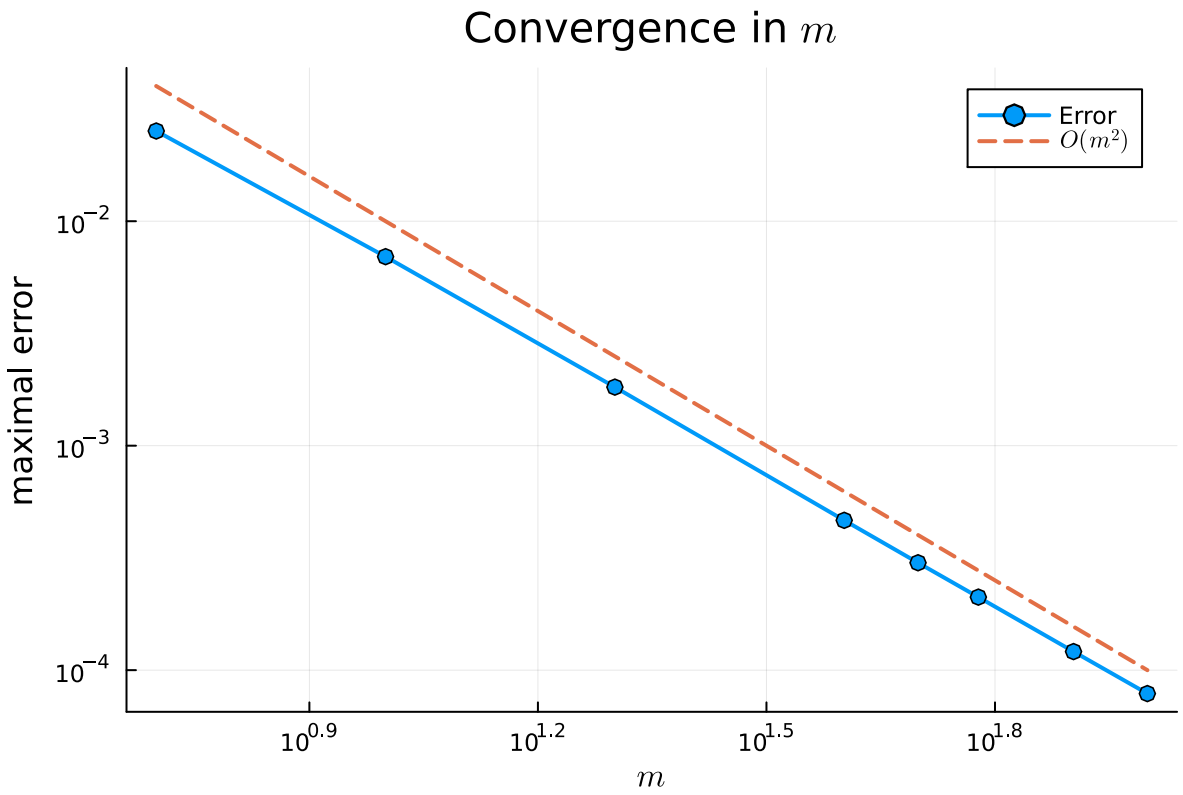
```julia
function sine_solution(m, x)
    # Compute the function value of the numerical solution
    # for given m and x by evaluating (27)

    result = 0.0
    for i in 1:m
        result += 2 * (-1)^(i+1) / i^3 * sin(i * x)
    end
    result
end
```

For different m this looks like:



We see that already for 3 and 5 $\sin$ functions we are pretty close to a reference solution with 300 $\sin$ basis functions.

Numerically we observe a **quadratic convergence** in a log-log plot:



Finite elements

In the context of piecewise linear polynomial interpolation (chapter 5) we already discussed the hat functions. Recall, that given a set of nodes $x_0 < x_1 < \cdots < x_n$ the hat functions are

$$H_i(x) = \begin{cases} \frac{x - x_{i-1}}{x_i - x_{i-1}} & \text{if } i > 0 \text{ and } x \in [x_{i-1}, x_i] \\ \frac{x_{i+1} - x}{x_{i+1} - x_i} & \text{if } i < n \text{ and } x \in [x_i, x_{i+1}] \\ 0 & \text{otherwise} \end{cases}$$

for $i = 0, \ldots, n$. In the context of Galerkin approximations these hat functions turn out to be a very useful (and widely applicable) choice for the basis functions $\varphi_i = H_i(x)$.

For our case, the weak form of the Dirichlet heat equation (21), a natural set of nodes is $0 = x_0 < x_1 < \cdots < x_n = L$ and for simplicity we will further assume the nodes to be equispaced with spacing $h = \frac{L}{n} = x_{i+1} - x_i$. In this setting the hat functions can be expressed as

$$H_i(x) = \begin{cases} \frac{x - x_{i-1}}{h} & \text{if } i > 0 \text{ and } x \in [x_{i-1}, x_i] \\ \frac{x_{i+1} - x}{h} & \text{if } i < n \text{ and } x \in [x_i, x_{i+1}] \\ 0 & \text{otherwise} \end{cases} \tag{28}$$

for $i = 0, \ldots, n$. Note, that since the hat functions $\varphi_i = H_i(x)$ satisfy the cardinality property $H_i(x_j) = \delta_{ij}$ this implies for the expansion (24)

$$u(x) = \sum_{j=1}^{m} c_j \varphi_j(x) = \sum_{i=1}^{n-1} u_i \, H_i(x) \tag{29}$$

where $u_i = u(x_i)$, i.e. the numerical solution at the nodal points. Note that the last sum in (29) is deliberately constructed to omit the hat functions H_0 and H_n, since their associated nodal point is either $0 = x_0$ or $L = x_n$. As a consequence these two functions are unable to satisfy the condition $0 = \varphi_j(0) = \varphi_j(L)$ underlying V_0.

The importance of the hat functions for Galerkin methods stems from the fact that each hat functions $H_i(x)$ is only non-zero in an interval $[x_{i-1}, x_{i+1}]$. Instead of considering integrals over the full domain $[0, L]$ when evaluating the Galerkin conditions (26) one only needs to compute integrations over a few subintervals.

As an example consider the evaluation of the elements of the matrix $\mathbf{A} \in \mathbb{R}^{m \times m} = \mathbb{R}^{(n-1)\times(n-1)}$. First note that H_i' is at most non-zero over the same interval as H_i, i.e. $[x_{i-1}, x_{i+1}]$. As a result the product $H_i' H_j'$ can only be non-zero in the intersection of $[x_{i-1}, x_{i+1}]$ and $[x_{j-1}, x_{j+1}]$. This intersection is empty whenever $i + 1 < j - 1 \Leftrightarrow i < j - 2$ or $i > j + 2$. Moreover it contains only a single element for $i = j - 2$ and $i = j + 2$. As a consequence

$$A_{i,i+d} = \int_0^L k \, H_i'(x) \, H_{i+d}'(x) \, dx = 0 \qquad \text{if } d \geq 2.$$

where $i = 1, \ldots, n-3$ and similarly by symmetry

$$A_{i+d,i} = \int_0^L k \, H_{i+d}'(x) \, H_i'(x) \, dx = 0 \qquad \text{if } d \geq 2.$$

again for $i = 1, \ldots, n-3$

Now we consider the remaining three cases, namely

$$
\begin{aligned}
A_{ii} &= \int_0^L k\, H_i'(x)\, H_i'(x)\, dx \\
&= \int_{x_{i-1}}^{x_i} k\, H_i'(x)\, H_i'(x)\, dx + \int_{x_i}^{x_{i+1}} k\, H_i'(x)\, H_i'(x)\, dx \\
&= \int_{x_{i-1}}^{x_i} k \cdot \frac{1}{h} \cdot \frac{1}{h}\, dx + \int_{x_i}^{x_{i+1}} k \cdot \frac{-1}{h} \cdot \frac{-1}{h}\, dx \\
&= \frac{k}{h^2} \left[\int_{x_{i-1}}^{x_i} 1\, dx + \int_{x_i}^{x_{i+1}} 1\, dx \right] \\
&= \frac{2k}{h}
\end{aligned}
$$

for $i = 1, \ldots, n-1$ as well as

$$
\begin{aligned}
A_{i+1,i} = A_{i,i+1} &= \int_0^L k\, H_i'(x)\, H_{i+1}'(x)\, dx \\
&= \int_{x_i}^{x_{i+1}} k\, H_i'(x)\, H_{i+1}'(x)\, dx \\
&= \int_{x_i}^{x_{i+1}} k \cdot \frac{-1}{h} \cdot \frac{1}{h}\, dx \\
&= -\frac{k}{h}.
\end{aligned}
$$

for $i = 1, \ldots, n-2$. We recover a tridiagonal matrix

$$
\mathbf{A} = \frac{k}{h} \begin{pmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & -1 & 2 & \ddots & \\ & & \ddots & \ddots & -1 \\ & & & -1 & 2 \end{pmatrix} \in \mathbb{R}^{(n-1)\times(n-1)} \tag{30}
$$

Finally we consider the elements of the vector $\mathbf{f}$. For this need to perform the two integrals

$$
f_i = \int_0^L f(x)\, H_i(x)\, dx = \int_{x_{i-1}}^{x_i} f(x)\, H_i(x)\, dx + \int_{x_i}^{x_{i+1}} f(x)\, H_i(x)\, dx
$$

for $i = 1, \ldots, n-1$. One can show that to a very good approximation one can replace $f(x)$ by the average value of the function over the respective integrals, i.e.

$$\int_{x_{i-1}}^{x_i} f(x)\, H_i(x)\, dx \approx \frac{f(x_{i-1}) + f(x_i)}{2} \int_{x_{i-1}}^{x_i} H_i(x)\, dx = \frac{f(x_{i-1}) + f(x_i)}{2} \cdot \frac{h}{2}$$
$$\int_{x_i}^{x_{i+1}} f(x)\, H_i(x)\, dx \approx \frac{f(x_i) + f(x_{i+1})}{2} \int_{x_i}^{x_{i+1}} H_i(x)\, dx = \frac{f(x_i) + f(x_{i+1})}{2} \cdot \frac{h}{2}.$$

In particular this approximation converges quadratically as $h \to 0$, which turns out to be exactly the order of approximation of the finite element method itself. Therefore putting in additional efforts to compute these integrals more accurately would not even improve the accuracy of our final result.

Putting these developments tothere we obtain for $i = 1, \ldots, n-1$ that

$$\begin{aligned} f_i &= \frac{f(x_i) + f(x_{i-1})}{2} \frac{h}{2} + \frac{f(x_{i+1}) + f(x_i)}{2} \frac{h}{2} \\ &= \frac{h}{4} \Big(f(x_{i-1}) + 2f(x_i) + f(x_{i+1}) \Big). \end{aligned} \tag{31}$$

From $\mathbf{A}$ and $\mathbf{f}$ the coefficients $\mathbf{u} = (u_1, u_2, \ldots, u_{n-1})^T$ in the expansion (29) is than obtained by solving the linear system

$$\mathbf{A}\mathbf{u} = \mathbf{f}.$$

Notably, due to the cardinality property of the hat functions $u_i = u(x_i)$, i.e. these coefficients are directly the numerical approximation to the solution $u(x)$ at the nodal points.

An implementation of this approch is given below:

```
heat_equation_1d_fem (generic function with 1 method)
```

```julia
function heat_equation_1d_fem(f, k, L, n)
    # f:  Function describing the external heat source
    # k:  thermal conductivity
    # L:  Length of the metal rod
    # N:  Number of nodal points for the finite element scheme

    h = L / n                   # Step size
    x = [i * h for i in 0:n]    # Nodal points (x₀, x₁, ..., xₙ)
    x_inner = x[2:n]            # Inner nodal points (x₁, ..., xₙ₋₁)


    # Build A and f: Note that these are a (n-1) × (n-1) matrix
    # respectively a vector of length (n-1)
    A = k/h * SymTridiagonal(2ones(n-1), -ones(n-2))
    f = h/4 * [f(x[i-1]) + 2f(x[i]) + f(x[i+1])
               for i in 2:n]   # Skip first nodal point, i.e. x[1] = x₀

    u = A \ f
    (; x=x_inner, u)
end
```

Let us return to the example we considered with the sine basis, i.e.

$$\begin{cases} -\dfrac{\partial^2 u}{\partial x^2}(x) = x & x \in (0, \pi) \\ u(0) = u(L) = 0, \end{cases}$$

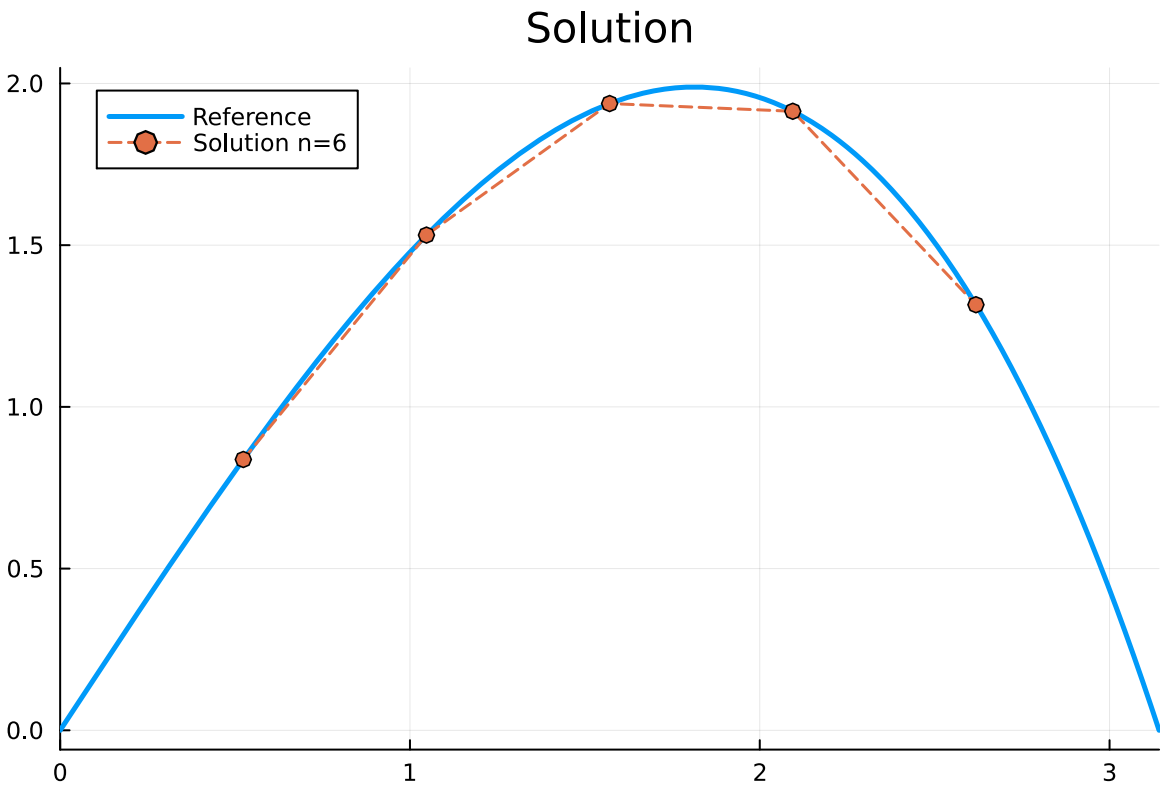
i.e. the Dirichlet heat equation (20) with $k = 1$, $L = \pi$ and $f(x) = x$.

As a reference solution we consider the solution using the sine basis with $m = 300$, which we know to be very accurate for this problem:

```
reference (generic function with 1 method)
```

```julia
reference(x) = sine_solution(300, x)
```

- n = 6




```julia
let
    # Parameters of the problem
    f(x) = x
    k = 1
    L = π

    # Plot the reference
    p = plot(reference; lw=2.5, label="Reference", title="Solution", xlims=(0, L))

    # Compute FEM solution and plot it
    result = heat_equation_1d_fem(f, k, L, n)
    plot!(p, result.x, result.u; lw=1.5, ls=:dash, label="Solution n=$n", mark=:o)
end
```

The finite element method is one of the most widely employed approaches in science and engineering to solve differential equations. With this short instroduction we only scratched the surface.

More information on the approach can be found for example in chapter 10.6 of Driscoll, Brown: *Fundamentals of Numerical Computation*.

Numerical analysis

1. Introduction

2. The Julia programming language
3. Revision and preliminaries
4. Root finding and fixed-point problems
5. Interpolation
6. Direct methods for linear systems
7. Iterative methods for linear systems
8. Eigenvalue problems
9. Numerical integration
10. Numerical differentiation
11. Initial value problems
12. Boundary value problems